반도체장비데이터관리 03

SQL: 집계함수

김 민 진 | muea.mj@gmail.com

SQL 자료형

데이터 형식 (Data Type)

- 특정 데이터를 받아들일 공간을 데이터의 유형별로 나누는 기준
- 설정한 형식이 아닌 다른 종류의 데이터가 들어오려고 하면 에러 발생
- SQL 데이터 형식에는 크게 숫자, 문자, 날짜형이 있음

- *<u>SQL에 자료형의 종류가 많은 이유</u>*

  : 각 행이 추가될 때마다 자료형만큼의 공간이 할당되기 때문에

  필요 이상으로 큰 자료형이 사용되면 저장공간이 낭비됨.

정수형

- 소수점이 없는 숫자, 즉 인원수, 가격, 수량 등에 많이 사용
- 정수형의 크기와 범위 :

자료형	Byte	SIGNED	UNSIGNED	개수
TINYINT	1	-128~127	0~255	2^8개
SMALLINT	2	-32,768 ~ 32,767	0 ~ 65,535	2^16개
MEDIUMINT	3	-8,388,608 ~ 8,388,607	0 ~ 16,777,215	2^24개
INT	4	-2,147,483,648 ~ 2,147,483,647	0 ~ 4,294,967,295	2^32개
BIGINT	8	-2^63 ~ 2^63 - 1	0 ~ 2^64 - 1	2^64개

https://velog.io/@impala/SQL-SQL-%EC%9E%90%EB%A3%8C%ED%98%95-%EC%A0%95%EB%A6%AC

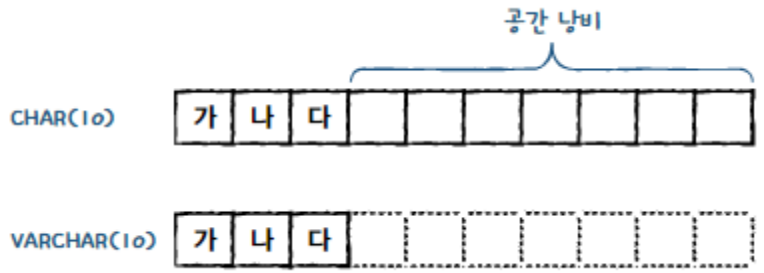
실수형

- 소숫점이 있는 숫자를 저장할 때 사용.

데이터 형식	바이트 수	설명
FLOAT	4	소수점 아래 7자리까지 표현
DOUBLE	8	소수점 아래 15자리까지 표현

문자형

- 글자를 저장하기 위해 사용.
- 입력할 최대 글자의 개수를 지정.



데이터 타입	바이트 수	설명
CHAR(n)	1 ~ 255	고정길이 문자형. n을 1부터 255까지 지정. CHARACTER의 약자이며, CHAR 이라고만 하면 CHAR(1)과 동일 CHAR(100) 인 경우 세 자리만 사용해도 나머지 97 자리를 할당. 성능은 CHAR이 VARCHAR보다 더 좋음.
VARCHAR(n)	1 ~ 65535	가변길이 문자형. n을 사용하면 1부터 65535까지 지정 Variable Character의 약자. VARCHAR(100) 인 경우 3글자를 저장하는 경우 3자리의 데이터 공간만 사용

https://inpa.tistory.com/entry/MYSQL-%F0%9F%93%9A-%EC%9E%90%EB%A3%8C%ED%98%95-%ED%83%80%EC%9E%85-%EC%A2%85%EB%A5%98-%EC%A0%95%EB%A6%AC

날짜형

- 날짜 및 시간을 저장할 때 사용.

자료형	형식	설명
DATE	YYYY-MM-DD	
TIME	HHH:MM:SS	
DATETIME	YYYY-MM-DD HHH:MM:SS	입력된 시간을 문자 그대로 저장, 8Byte
TIMESTAMP	YYYY-MM-DD HHH:MM:SS	MYSQL이 설치된 컴퓨터의 시간대를 기준으로 저장, 4Byte

- DATETIME : 시간 데이터를 그대로 저장할 때 사용(역사적인 시간 등), 1000~9999년
- TIMESTAMP : 시차를 반영한 시간을 저장할 때 사용(게시물 작성 시간 등), 1970~2038년
- TIMESTAMP형 데이터의 경우 아무것도 적지 않으면 자동으로 현재시간 저장

https://velog.io/@impala/SQL-SQL-%EC%9E%90%EB%A3%8C%ED%98%95-%EC%A0%95%EB%A6%AC

그 밖에

❖논리형

- 참(true) 또는 거짓(false)의 두 가지 상태값만을 저장하는 자료형

타입	설명	예시
BOOLEAN	참/거짓	TRUE, FALSE

❖NULL

- 값이 없음을 나타내는 특수 값

개념	설명
NULL	값이 없음 (비어있음)

데이터 유형별 포인트!

- 숫자형 데이터 (정수형과 실수형)
  - 계산이 가능하다.
  - 따옴표를 쓰지 않는다.

```
WHERE price >= 20000
```

- 문자형 데이터
  - 반드시 따옴표를 사용한다.
  - 비교가 가능하다. (=, LIKE 사용)

```
WHERE name = '김지훈'
```

데이터 유형별 포인트!

- 날짜형 데이터
  - 문자열처럼 보이지만 비교/정렬 가능하다.
  - 최신/오래된 데이터 찾을 때 핵심!

```
WHERE use_date >= '2024-03-10'
```

- NULL
  - 값이 없기 때문에 비교연산자를 쓸 수 없다. (= NULL 안됨)
  - 반드시 IS NULL / IS NOT NULL을 써야 한다.

```
WHERE issue_report IS NULL
```

SQL에서 괄호의 사용

- 여러 요소를 "하나의 단위"로 묶는 문법
  - SQL이 구조를 이해할 수 있도록 도와줌
  - 괄호가 없으면 해석이 달라지거나 오류 발생

→ *괄호는 묶음!*

```
WHERE (price > 10000 AND stock > 0)
```

→ 조건의 우선순위 제어

```
WHERE department IN ('개발', '영업')
```

→ 여러 값을 하나의 묶음으로 표현

```
COUNT(*)
SUM(price)
```

→ 괄호 안 = 함수의 입력값

```
WHERE price > (SELECT AVG(price) FROM Book)
```

→ 쿼리 결과를 하나의 값처럼 사용

연산자 IN / NOT IN과 괄호

- IN 뒤에는 "하나의 집합(값들의 묶음)"이 와야 한다

```
WHERE department IN '개발', '영업'
```

→ X 오류 발생

```
WHERE department = '개발, 영업'
```

→ X 하나의 문자열로 인식됨

```
WHERE department IN ('개발', '영업')
```

→ O 값들의 묶음으로 인식

```
WHERE department IN (
    SELECT department
    FROM Employee
)
```

→ O 괄호 안에 쿼리 결과를 묶음으로 인식

연산자 IN / NOT IN과 등호(=)

- IN은 "묶음 안에 속성이 포함되는지" 검사

```
WHERE 속성 IN 묶음
```

- 등호(=)는 "속성이 값과 동일한지" 검사
  - 등호 뒤에는 단일한 값이 와야 한다. (묶음 X)

```
WHERE 속성 = 값
```

집계함수

별칭 붙이기: AS

❖별칭(Alias)

- 속성(칼럼), 테이블 등에 별도의 이름을 붙여서 질의 결과를 만드는 것

속성명 AS 별칭

```
select name as 이름
from equipmentuser;
```

'name'이란 속성을 결과에 표시할 때, 컬럼을 '이름'으로 표시함

이름
김지훈
박서윤
이준호
김민아
정예린

Query 1

```
1  select last_name as 성,
2         first_name as 직원_이름,
3         gender as 성별
4
5  from employees limit 5;
```

Result Grid Filter Rows:

성	직원_이름	성별
단	군	남자
고	주몽	남자
관	창	남자
이	사부	남자
최	무선	남자

별칭 붙이기: AS

❖별칭(Alias)

- 속성(칼럼), 테이블 등에 별도의 이름을 붙여서 질의 결과를 만드는 것

속성명 AS 별칭

```
select name as '이름'
from equipmentuser;
```

별칭은 작은 따옴표로 표시할 수도 있음

속성명 AS 별칭

```
select name as '사용자 이름'
from equipmentuser;
```

별칭에 공백이 들어갈 때는 반드시 작은 따옴표로 감싸야 함.

사용자 이름
김지훈
박서윤
이준호
김민아
정예린

별칭 붙이기: AS

❖별칭(Alias)

- 속성(칼럼), 테이블 등에 별도의 이름을 붙여서 질의 결과를 만드는 것

```
select name as '이름'
from equipmentuser;
```

이름
김지훈
박서윤
이준호
김민아
정예린

```
select name as 이름
from equipmentuser;
```

```
select name as '사용자 이름'
from equipmentuser;
```

사용자 이름
김지훈
박서윤
이준호
김민아
정예린

집계 함수

- 간단한 명령을 사용하여 대규모 데이터 세트를 요약하는 도구
- 값의 집합을 처리하고, **단일 요약된 결과를 반환함**

함수	설명	사용 예시 (SQL)
COUNT()	행의 개수를 계산	SELECT COUNT(*) FROM Orders; # 전체 주문 수
SUM()	값들의 합계 계산	SELECT SUM(saleprice) FROM Orders; # 총 판매 금액
AVG()	값들의 평균 계산	SELECT AVG(price) FROM Book; # 평균 도서 가격
MAX()	가장 큰 값 반환	SELECT MAX(price) FROM Book; # 최고가 도서
MIN()	가장 작은 값 반환	SELECT MIN(price) FROM Book; # 최저가 도서

집계 함수: COUNT

- **정의:** 행 수를 반환합니다.

- **변형:**
  - **COUNT(*)** : NULL을 포함한 모든 행.
  - **COUNT(column)** : NULL이 아닌 값만.
  - **COUNT(DISTINCT column)** : 고유한 NULL이 아닌 항목 수.

- **예제:**

질의 3-18

마당서점의 도서 판매 건수를 구하시오.

```
SELECT  COUNT(*)
FROM    Orders;
```

COUNT(*)
10

집계 함수: SUM

- **정의**: 숫자 값의 합계를 계산합니다.
- **NULL 처리**: 무시합니다.
- **예제**:

```
SELECT SUM(amount) AS TotalRevenue
FROM Sales;
```

https://leapcell.io/blog/ko/sql-jibgye-hamsu-ihae

집계 함수: SUM

질의 3-15

고객이 주문한 도서의 총판매액을 구하시오.

```
SELECT   SUM(saleprice)
FROM     Orders;
```

SUM(saleprice)
118000

- 속성에 별칭 붙이기: AS

```
SELECT   SUM(saleprice) AS 총매출
FROM     Orders;
```

총매출
118000

집계 함수: AVG

- **정의**: 숫자 값의 평균을 계산합니다.
- **계산**: NULL이 아닌 값의 `SUM / COUNT`.
- **예제**:

```
SELECT AVG(salary) AS AvgSalary
FROM Employees;
```

https://leapcell.io/blog/ko/sql-jibgye-hamsu-ihae

집계 함수: MIN / MAX

- **정의**:
  - `MIN()`: 열에서 가장 작은 값입니다.
  - `MAX()`: 열에서 가장 큰 값입니다.
- **예제**:

```sql
SELECT MIN(salary) AS LowestSalary,
       MAX(salary) AS HighestSalary
FROM Employees;
```

https://leapcell.io/blog/ko/sql-jibgye-hamsu-ihae

질의 3-16

2번 김연아 고객이 주문한 도서의 총판매액을 구하시오.

```
SELECT  SUM(saleprice) AS 총매출
FROM    Orders
WHERE   custid=2;
```

총매출
15000

질의 3-17

고객이 주문한 도서의 총판매액, 평균값, 최저가, 최고가를 구하시오.

```
SELECT  SUM(saleprice) AS Total,
        AVG(saleprice) AS Average,
        MIN(saleprice) AS Minimum,
        MAX(saleprice) AS Maximum
FROM    Orders;
```

Total	Average	Minimum	Maximum
118000	11800.0000	6000	21000

끝!